```
Section 2
Warm Up Questions
```

1. What is the difference between == and is in Python? Give one example where they produce different results.

== checks whether two objects have the same value, while is checks whether they are the exact same object in memory. This is == for comparing contents and is for comparing identity. In example below Both lists contain the same values, but they are stored as different objects.

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)
print(a is b)
```

```
True
False
```

1. Why does 0.1 + 0.2 == 0.3 return False? What is the correct way to compare floats?

Computers store floating-point numbers in binary, and some decimal values like 0.1 and 0.2 cannot be represented exactly. This creates tiny rounding errors behind the scenes. As a result, 0.1 + 0.2 becomes something very close to 0.3, but not exactly equal because of floating-point precision limitations.

```
a = 0.1 + 0.2
b = 0.3
print(a == b)
print(abs(a - b) < 1e-9)

#Modern Method
import math
math.isclose(0.1 + 0.2, 0.3)
```

```
False
True

True
```

1. Explain the LEGB rule. In what order does Python resolve a variable name?

LEGB describes the order Python follows when searching for a variable: Local – inside the current function. Enclosing – inside any outer functions. Global – at the module level. Built-in – Python's built-in names like len() or print().

For example, if a variable exists both locally and globally, Python uses the local one first. LEGB helps avoid confusion when variables share names across different scopes and explains why some variables are accessible while others raise NameError.

1. What is a mutable default argument bug? How do you fix it?

A mutable default argument is created only once when the function is defined, not each time it is called. This means changes persist between calls, often causing unexpected behavior. def f(x=[]) is a bug — use def f(x=None): if x is None: x = [] Using None creates a fresh list every call. This prevents accidental sharing of data between function invocations.

```
def add(item , lst=None):
    if lst is None:
        lst=[]
        lst.append(item)
        return lst
add(1)
add(2)
```

```
[2]
```

1. What does and return when all values are truthy? What does or return when all values are falsy?

Python's and and or operators return operands themselves, not necessarily True or False. With and, if all values are truthy, Python returns the last operand. With or, if all values are falsy, Python returns the last falsy operand.

```
print("Hello" and 42 and [1, 2])
print(None or "" or [])
```

```
[1, 2]
[]
```

1. What is the difference between // and /? What does -7 // 2 return and why?

/ performs normal division and returns a floating-point result: 7 / 2 # 3.5

// performs floor division, meaning it rounds the result down to the nearest integer: 7 // 2 # 3

For negative numbers: -7 // 2 # -4 Python rounds toward negative infinity, not toward zero. 3.5 floored becomes -4.

1. What does for-else do in Python? When does the else block run?

The else block in a for loop runs only if the loop completes normally without encountering a break statement. Since 5 is never found, the loop finishes naturally and the else block executes. This pattern is commonly used in search operations where require special handling when an item is not found.

```
for n in [1, 2, 3]:
    if n == 2:
        break
else:
    print("Not found")

for n in [1, 2, 3]:
```

```
    if n == 5:
        break
else:
    print("Not found")
```

```
Not found
```

1. Why should you never use str += inside a loop? What should you use instead?

Strings are immutable in Python, meaning every += operation creates a brand-new string. In a loop, this leads to repeated copying and poor performance, especially with large amounts of text.

A better approach is collecting pieces in a list and joining them once. This is significantly faster and more memory-efficient.

```
words="this is a dog"
result1 = ""
for word in words:
    result1 += word

parts = []
for word in words:
    parts.append(word)
print(parts)
result2 = "".join(parts)

print(result1)
print(result2)
```

```
['t', 'h', 'i', 's', ' ', 'i', 's', ' ', 'a', ' ', 'd', 'o', 'g']
this is a dog
this is a dog
```

1. What is the difference between a generator and a list comprehension?

A list comprehension creates and stores all results in memory immediately A generator expression produces values one at a time as needed

Generators are more memory-efficient because they don't store the entire sequence at once. They're ideal for processing large datasets, while list comprehensions are often faster when you need all values available immediately.

```
squares1 = [x*x for x in range(10)]
squares2 = (x*x for x in range(10))
print(squares1)
print(squares2)
#To get value from generator use next()
print(next(squares2))
print(next(squares2))
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
<generator object <genexpr> at 0x0000025D1A067780>
0
1
```

What does @functools.wraps(func) do, and why is it mandatory in decorators?

Answer- The @functools.wraps(func) decorator is mandatory in decorators because it ensures that the metadata of the original function being decorated is preserved. This includes attributes like the function's name, docstring, module name, and signature. Without wraps, the decorated function's metadata would be incorrect, showing the wrapper's name and docstring instead of the original function's. This can lead to confusion, especially if the same wrapper function is used for different functions, and can affect debugging and documentation.

Rapid-Fire Questions

```
What does type(True) return?
Answer: bool
Is bool a subclass of int in Python?
Answer: Yes
What is the output of True + True + True?
Answer: 3
What Python version introduced the walrus operator :=?
Answer: Python 3.8
What Python version introduced match-case?
Answer: Python 3.10
What is the CPython integer cache range?
Answer: -5 to 256
What does id() return?
Answer: The unique identity of an object
Name three falsy values in Python.
Answer: False, None, 0
What operator checks membership in a collection?
Answer: in
What does **kwargs capture in a function?
Answer: Extra keyword arguments as a dictionary
```

Fill in the Blanks

```
# 1. To compare floats correctly:
abs(a - b) < tolerance

# 2. To swap two variables in one line:
a, b = b, a

# 3. To check if a variable is None (idiomatic way):
if result is None:

# 4. The correct fix for mutable default argument:
```

```
def append_item(item, lst=None):
    if lst is None:
        lst = []

# 5. To build a string from a list efficiently:
result = "".join(words)

# 6. The walrus operator in a while loop:
while chunk := f.read(1024):
    process(chunk)

# 7. To preserve function metadata in a decorator:
@functools.wraps(func)
def wrapper(*args, **kwargs):

# 8. Floor division of -9 by 2 equals:
-5

# 9. To get both index and value in a loop:
for i, v in enumerate(my_list):

10. 'and' returns the first falsy value, or the last value if all
truthy.

Section 03
Predict the Output

Easy Level
# E1 — Your prediction:
3
1
3.3333333333333335

__________
x = 10
y = 3
print(x // y)
print(x % y)
print(x / y)

# E2 — Your prediction:
@LICE

___________
name = "  Alice  "
print(name.strip().upper().replace('A', '@'))

# E3 — Your prediction:
[1, 2, 3, 4]
True

___________
a = [1, 2, 3]
```

```
b = a
b.append(4)
print(a)
print(a is b)

# E4 — Your prediction:
False True False True False

___________
print(bool(""), bool(" "), bool(0), bool([0]), bool(None))


 Intermediate Level
# I1 — Your prediction: ___________
print(0 and "hello") # Your prediction: __0_________
print("" or "default") # Your prediction: _default__________
print(None or [] or 0) #Your prediction: ___0________
print("a" and "b" and "c") # Your prediction: __c_________

# I2 — Your prediction:
positive , negative

___________
x = 5
result = "positive" if x > 0 else "negative" if x < 0 else "zero"
print(result)
y = -3
result2 = "positive" if y > 0 else "negative" if y < 0 else "zero"
print(result2)

# I3 — Your prediction:
World
!dlroW ,olleH
Hlo ol!

___________
s = "Hello, World!"
print(s[7:12])
print(s[::-1])
print(s[::2])

# I4 — Your prediction:
6
9
20

___________
def f(x, multiplier=2):
    return x * multiplier
print(f(3))
print(f(3, 3))
print(f(multiplier=5, x=4))
```

```
 Tricky Cases
# T1 — Your prediction:
-4
1
-4
-1

___________
print(-7 // 2)
print(-7 % 2)
print(7 // -2)
print(7 % -2)

# T2 — Your prediction:
512
64

___________
print(2 ** 3 ** 2)    # right-associative exponentiation
print((2 ** 3) ** 2)

# T3 — Your prediction:
['python']
['python', 'ml']
['python', 'ml', 'ai']

__________
def add_tag(item, tags=[]):
    tags.append(item)
    return tags
print(add_tag("python"))
print(add_tag("ml"))
print(add_tag("ai"))

# T4 — Your prediction:
[2, 2, 2]

___________
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])

print(-7 // 2)
print(-7 % 2)
print(7 // -2)
print(7 % -2)

-4
1
-4
-1
```

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])
```

```
[2, 2, 2]
```

Hidden Behavior Questions

```
# H1 – Integer caching: Your prediction: __True True_________
a = 256
b = 256
print(a is b)
c = 257
d = 257
print(c is d)
```

```
True
False
```

```
# H2 – Boolean arithmetic: Your prediction: ___2 ,10 ,3 , 1 0________
print(True + True)
print(True * 10)
print(sum([True, False, True, True, False]))
print(int(True), int(False))
```

```
2
10
3
1 0
```

```
# H3 – for-else: Your prediction: _All Even*5  Found Odd __________
for n in [2, 4, 6, 8]:
    if n % 2 != 0:
        print("Found odd")
        break
    else:
        print("All even")

for n in [2, 3, 6, 8]:
    if n % 2 != 0:
        print("Found odd")
        break
    else:
        print("All even")
```

```
All even
All even
```

```
All even
All even
All even
Found odd
```

Section- 04 Debugging Challenge Zone

```
Syntax Error
```

```
def greet(name):
    message = f"Hello, {name}!"
    return message

print(greet("Sonam"))
```

```
Hello, Sonam!
```

```
for i in range(5):
    if i % 2 == 0:
        print(i, "is even")
```

```
0 is even
2 is even
4 is even
```

Logical Error

```
def calculate_fee(amount):
    fee = amount * 2 / 100
    if fee < 5:
        fee = 5       # BUG: What is wrong here?
    if fee > 200:
        fee = 200     # BUG: And here?
    return fee
print(calculate_fee(100))    # Expected: 5 (minimum applies)
print(calculate_fee(5000))   # Expected: 100
print(calculate_fee(15000))  # Expected: 200
```

```
5
100.0
200
```

```
def classify_grade(score):
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    elif score >= 70:
        grade = "C"
    elif score >= 60:
```

```
        grade = "D"
    else:
        grade = "F"           # BUG: Where is the error?
    return grade
print(classify_grade(95))  # Should be A
print(classify_grade(75))  # Should be C
print(classify_grade(45))  # Should be F
```

```
A
C
F
```

```
def count_vowels(text):
    count = 0
    for char in text:
        if char in "aeiouAEIOU":   # BUG: What's missing?
            count += 1
    return count
print(count_vowels("Hello World"))   # Expected: 3
print(count_vowels("APPLE"))         # Expected: 2
```

```
3
2
```

Runtime Error

```
#Division by zero error fix
def divide(a, b):
    if b == 0:
        return None
    return a / b

results = [divide(10, x) for x in [2, 5, 0, 1]]
print(results)
```

```
[5.0, 2.0, None, 10.0]
```

```
#key error fix
data = {"name": "Alice", "age": 25}
print(f"Name: {data['name']}, Age: {data['age']}")
```

```
Name: Alice, Age: 25
```

```
#UnboundLocalError fix
counter = 0

def increment():
    global counter
    counter += 1
    return counter
```

```
print(increment())
print(increment())
```

```
1
2
```

Find the Hidden Isuues

```
#logic Error Fix
numbers = [1, -2, 3, -4, 5, -6, 7]
for num in numbers[:]:
    if num < 0:
        numbers.remove(num)
print(numbers)
```

```
[1, 3, 5, 7]
```

WHY IT FAILS:

try to modifying the list while iterating over it. When an element is removed, Python shifts remaining elements left. The loop then skips the element immediately after the removed one.

```
#logic error fix
multipliers = []

for i in range(3):
    multipliers.append(lambda x, i=i: x * i)

print([f(10) for f in multipliers])
```

```
[0, 10, 20]
```

SECTION 05

Edge Cases & Hidden Scenarios

```
def average(numbers):
    """Return the average of a list of numbers."""

    if numbers is None:
        raise TypeError("numbers cannot be None")

    if not isinstance(numbers, list):
        raise TypeError("numbers must be a list")

    if len(numbers) == 0:
        raise ValueError("cannot calculate average of an empty list")

    for item in numbers:
        if not isinstance(item, (int, float)):
```

```
            raise TypeError(
                f"all items must be numeric"
            )

    return sum(numbers) / len(numbers)

print(average([5]))
try:
    average(["a", "b"])
except TypeError as e:
    print(e)
```

```
5.0
all items must be numeric
```

```
def is_valid_password(pwd):
    # Length must be between 8 and 20 characters
    if len(pwd) < 8 or len(pwd) > 20:
        return False

    has_upper = any(c.isupper() for c in pwd)
    has_lower = any(c.islower() for c in pwd)
    has_digit = any(c.isdigit() for c in pwd)
    has_special = any(not c.isalnum() for c in pwd)

    return has_upper and has_lower and has_digit and has_special

print(is_valid_password(""))            # False
print(is_valid_password("abc"))         # False
print(is_valid_password("a" * 21))      # False
print(is_valid_password("Abcdefg1!"))   # True
print(is_valid_password("ABCDEFG1!"))   # False
print(is_valid_password("abcdefg1!"))   # False
print(is_valid_password("Abcdefgh!"))   # False
print(is_valid_password("Abcdefg12"))   # False
```

```
False
False
False
True
False
False
False
False
```

```
def validate_student_age(age):
    if age is None:
        raise TypeError("Age cannot be None")

    if isinstance(age, str):
        if age.isdigit():
```

```
            age = int(age)
        else:
            raise TypeError("Age must be a number")

    if not isinstance(age, int):
        raise TypeError("Age must be an integer")

    return 5 <= age <= 18

validate_student_age("15")      # True
validate_student_age(-1)        # False
# validate_student_age(None)      # TypeError
# validate_student_age("fifteen") # TypeError
# validate_student_age(15.7)      # TypeError
```

```
False
```

```
# 1. Time Complexity: O(n²)

# 2. Space Complexity: O(k)
#    where k = number of duplicate items stored

# Optimized O(n) solution:

def find_duplicates(items):
    seen = set()
    duplicates = set()

    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)

    return list(duplicates)

print(find_duplicates([1, 2, 3, 2, 4, 5, 1, 6, 3]))
```

```
[1, 2, 3]
```

```
SECTION 06
 Industry-Based Scenarios

Banking System - fee calculator for upi payment

def calculate_upi_fee(amount):
    """
    Calculate UPI transaction fee.
    Returns: fee as float, or None if input is invalid.
    """
    try:
```

```
        amount = float(amount)
    except (TypeError, ValueError):
        print("Error: Invalid amount")
        return None

    if amount <= 0:
        print("Error: Amount must be positive")
        return None

    if amount < 100:
        return "free"

    fee = amount * 0.018
    fee = max(2, min(fee, 1000))

    return round(fee, 2)


# Test cases
test_cases = [
    (50, "free"),
    (100, 2.0),
    (500, 9.0),
    (70000, 1000.0),
    (499.99, 9.0),
    ("500", 9.0),
    (-100, None),
    (0, None),
]

for amount, expected in test_cases:
    result = calculate_upi_fee(amount)
    status = "✓" if result == expected else "✗"
    print(f"{status} calculate_upi_fee({amount}) = {result} |
expected: {expected}")
```

```
✓ calculate_upi_fee(50) = free | expected: free
✓ calculate_upi_fee(100) = 2 | expected: 2.0
✓ calculate_upi_fee(500) = 9.0 | expected: 9.0
✓ calculate_upi_fee(70000) = 1000 | expected: 1000.0
✓ calculate_upi_fee(499.99) = 9.0 | expected: 9.0
✓ calculate_upi_fee(500) = 9.0 | expected: 9.0
Error: Amount must be positive
✓ calculate_upi_fee(-100) = None | expected: None
Error: Amount must be positive
✓ calculate_upi_fee(0) = None | expected: None
```

```
Healthcare System- a BMI calculator and health risk classifier.

def calculate_bmi(weight_kg, height_m):
    try:
```

```
        weight_kg = float(weight_kg)
        height_m = float(height_m)
    except (TypeError, ValueError):
        print("Error: Weight and height must be numbers.")
        return None

    if not (1 <= weight_kg <= 500):
        print("Error: Weight must be between 1 and 500 kg.")
        return None

    if not (0.5 <= height_m <= 2.5):
        print("Error: Height must be between 0.5 and 2.5 m.")
        return None

    bmi = round(weight_kg / (height_m ** 2), 1)

    if bmi < 18.5:
        category = "Underweight"
        risk_level = "Moderate"
    elif bmi < 25:
        category = "Normal"
        risk_level = "Low"
    elif bmi < 30:
        category = "Overweight"
        risk_level = "Elevated"
    else:
        category = "Obese"
        risk_level = "High"

    return {
        "bmi": bmi,
        "category": category,
        "risk_level": risk_level
    }


# Test cases
print(calculate_bmi(70, 1.75))
print(calculate_bmi(50, 1.75))
print(calculate_bmi(100, 1.75))
print(calculate_bmi(-5, 1.75))
print(calculate_bmi(70, 0))
print(calculate_bmi("seventy", 1.75))
```

```
{'bmi': 22.9, 'category': 'Normal', 'risk_level': 'Low'}
{'bmi': 16.3, 'category': 'Underweight', 'risk_level': 'Moderate'}
{'bmi': 32.7, 'category': 'Obese', 'risk_level': 'High'}
Error: Weight must be between 1 and 500 kg.
None
Error: Height must be between 0.5 and 2.5 m.
```

```
None
Error: Weight and height must be numbers.
None

SECTION 07
 Think Like an Engineer
```

Scenario A: A function get_user_data(user_id) fetches a user's data from a database and returns a dict.

1. What could go wrong? (Failure Modes) 1.Invalid user_id (None, string, negative value). 2.User not found in the database. 3.Database connection failure. 4.Corrupted/incomplete user data returned. 5.Permission/authentication error accessing the DB.
1. Assumptions the Design is Making-

1 user_id is valid and correctly formatted. 2 The user exists in the database. 3 The database is always available. 4 The query will complete successfully. 5 Returned data is a valid dictionary. 6 Network connectivity is stable.

1. Behavior at Scale (1 Million Calls/Hour) ≈ 278 requests/second continuously. Database may become a bottleneck. Increased latency and timeouts. Connection pool exhaustion. Higher infrastructure costs. Risk of cascading failures if DB slows down.

Improvements: Caching (e.g., frequently accessed users) Read replicas Load balancing Rate limiting Monitoring and alerting

```
Scenario B: A loop processes 10,000 order records. For each order, it
builds a report string using
+=
```

1. What is the time complexity of string += in a loop? Why? Each += creates a new string because strings are immutable. Existing content must be copied every time.

For n records:Time Complexity: $O(n^2)$

1. How much memory could this consume? 10,000 records × 1 KB each = 10 MB final report size. During concatenation, many intermediate strings are created and discarded.

Approximate temporary data copied: 1+2+3+...+10000

≈ 50 million KB copied over the entire process

≈ 50 GB of string copying work (even though the final output is only ~10 MB

```
3. What is the correct approach?

Use a list and ''.join()
Complexity:

Time: O(n)
Space: O(n)
```

```
join() allocates the final string once.
```

SECTION 08 Hands-On Coding Practice

```
Begineer - to find whether sting is palindrome or not

def is_palindrome(text):
    cleaned = text.lower().replace(" ", "")
    return cleaned == cleaned[::-1]

assert is_palindrome("racecar") == True
assert is_palindrome("hello") == False
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("") == True

print("B2 passed")

B2 passed

Begineer - to convert temperature into fahrenheit and kelvin

def convert_temperature(celsius):
    fahrenheit = (celsius * 9/5) + 32
    kelvin = celsius + 273.15
    return (fahrenheit, kelvin)

# Tests
assert convert_temperature(0) == (32.0, 273.15)
assert convert_temperature(100) == (212.0, 373.15)

print("B1 passed ✓")

B1 passed ✓

begineer - fizz buzz problem

fizzbuzz = [
    "FizzBuzz" if i % 15 == 0
    else "Fizz" if i % 3 == 0
    else "Buzz" if i % 5 == 0
    else i
    for i in range(1, 31)
]

# Test
assert fizzbuzz[2] == "Fizz"       # 3
assert fizzbuzz[4] == "Buzz"       # 5
assert fizzbuzz[14] == "FizzBuzz"  # 15
assert fizzbuzz[6] == 7
```

```
print("B3 passed ✓", fizzbuzz[:15])
```

```
B3 passed ✓ [1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz',
11, 'Fizz', 13, 14, 'FizzBuzz']
```

Inermediate - Write a decorator timer that measures and prints execution time

```
import functools
import time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()

        print(f"{func.__name__} took {end - start:.4f}s")
        return result

    return wrapper


@timer
def slow_sum(n):
    """Sum numbers from 0 to n."""
    return sum(range(n))


result = slow_sum(10_000_000)
print(f"Result: {result}")
print(f"Function name preserved: {slow_sum.__name__}")
```

```
slow_sum took 0.3048s
Result: 49999995000000
Function name preserved: slow_sum
```

Intermediate - generator for fibonacci sequence

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b


# Tests
fib_list = list(fibonacci(10))
assert fib_list == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34], f"Got
{fib_list}"
```

```
# Find first Fibonacci > 1000
first_over_1000 = next(x for x in fibonacci(20) if x > 1000)

print(f"First Fibonacci > 1000: {first_over_1000}")
print("I3 passed ✓")
```

```
First Fibonacci > 1000: 1597
I3 passed ✓
```

```
SECTION 09
 Scenario-Based Coding Tasks

SCENARIO 1: Student Report Card Generator

def generate_report(students):
    if not students:
        return []

    report = []

    for student in students:
        marks = student.get("marks", {})

        # Validate marks
        if not marks or any(not (0 <= m <= 100) for m in
marks.values()):
            print(f"Invalid marks for {student.get('name',
'Unknown')}")
            continue

        total = sum(marks.values())
        average = round(total / len(marks), 2)

        if average >= 90:
            grade = "A"
        elif average >= 80:
            grade = "B"
        elif average >= 70:
            grade = "C"
        elif average >= 60:
            grade = "D"
        else:
            grade = "F"

        report.append({
            "name": student["name"],
            "total": total,
            "average": average,
            "grade": grade
        })
```

```
    # Rank by average (highest first)
    report.sort(key=lambda x: x["average"], reverse=True)

    for rank, record in enumerate(report, start=1):
        record["rank"] = rank

    return report


students = [
    {"name": "Alice",   "marks": {"math": 85, "science": 92,
"english": 78}},
    {"name": "Bob",     "marks": {"math": 72, "science": 68,
"english": 80}},
    {"name": "Charlie", "marks": {"math": 95, "science": 88,
"english": 91}},
    {"name": "Priya",   "marks": {"math": 60, "science": 55,
"english": 70}},
]

for record in generate_report(students):
    print(record)
```

```
{'name': 'Charlie', 'total': 274, 'average': 91.33, 'grade': 'A',
'rank': 1}
{'name': 'Alice', 'total': 255, 'average': 85.0, 'grade': 'B', 'rank':
2}
{'name': 'Bob', 'total': 220, 'average': 73.33, 'grade': 'C', 'rank':
3}
{'name': 'Priya', 'total': 185, 'average': 61.67, 'grade': 'D',
'rank': 4}
```

scenerio 2- word frequency analyzer

```
import re
sample_text = """
Python is a versatile programming language. Python is used for web
development,
data science, machine learning, and automation. Python code is clean
and readable.
Many data scientists use Python for building machine learning models.
"""

def analyze_text(text):
    words = [w.lower() for w in re.findall(r"\w+", text) if len(w) >=
3]

    freq = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1
```

```
    return {
        "word_count": len(words),
        "unique_words": len(set(words)),
        "top_5_words": sorted(freq.items(), key=lambda x: x[1],
reverse=True)[:5],
        "avg_word_length": round(sum(len(w) for w in words) /
len(words), 2),
        "longest_word": max(words, key=len)
    }


result = analyze_text(sample_text)

for key, value in result.items():
    print(f"{key}: {value}")
```

```
word_count: 30
unique_words: 22
top_5_words: [('python', 4), ('for', 2), ('data', 2), ('machine', 2),
('learning', 2)]
avg_word_length: 6.17
longest_word: programming
```

```
SECTION 10
 Production Thinking Challenges

Scalability
1. The following function is called 10,000 times per second in
production. What would you change?
Don't recreate the dictionary on every call.
Create it once globally.
Faster and uses less memory at 10,000 calls/sec.

TAX_RATES = {"IN": 18, "US": 8, "UK": 20, "DE": 19}

def get_tax_rate(country):
    return TAX_RATES.get(country, 0)

2. A function builds a report for each of 500,000 users. It currently
uses a list to accumulate results and appends one item
per user. What is the memory concern, and what is the solution?
Concern: Storing all reports in a list can consume a lot of memory.
Solution: Use a generator (yield) or process records in batches.

def generate_reports(users):
    for user in users:
        yield create_report(user)

Reliability
3. What is wrong with this error handling?
```

```
Problems:
Hides all errors.
Makes debugging difficult.
Payment failures go unnoticed
better way

try:
    result = process_payment(order)
except Exception as e:
    print(f"Payment failed: {e}")

Readability & Maintainability
5. Rewrite this unreadable one-liner as clean, readable code:

r = {}

for n in range(1, 11):
    value = n * (n + 1) // 2  # triangular number

    if value % 3 == 0:
        r[n] = value

print(r)

SECTION 11
 Code Improvement Exercises

#Improve 1
from typing import List

def double_positive_numbers(numbers: List[float]) -> List[float]:
    """
    Return a list containing double the value
    of all positive numbers.
    """
    if numbers is None:
        return []

    result = []

    for num in numbers:
        if not isinstance(num, (int, float)):
            continue
        if num > 0:
            result.append(num * 2)

    return result

# Example
print(double_positive_numbers([1, -2, 3, "a", 4]))
# [2, 6, 8]
```

```
[2, 6, 8]
```

```
#Improve 2
from typing import List, Optional

def add_to_cart(item: str, cart: Optional[List[str]] = None) ->
List[str]:
    """
    Add an item to a cart and return the updated cart.
    """
    if not item:
        raise ValueError("Item cannot be empty")

    if cart is None:
        cart = []

    cart.append(item)
    return cart

# Example
cart1 = add_to_cart("apple")
cart2 = add_to_cart("banana")

print(cart1)  # ['apple']
print(cart2)  # ['banana']
```

```
['apple']
['banana']
```

```
#improve 3
def has_duplicate_fast(items):
    seen = set()

    for item in items:
        if item in seen:
            return True
        seen.add(item)

    return False
print(has_duplicate_fast([1, 2, 3, 4]))      # False
print(has_duplicate_fast([1, 2, 3, 2]))      # True
```

```
False
True
```